ANSWERS

1. a) $\overline{A}\,\overline{B}+A(\overline{B}+\overline{C})+C=\overline{A}\,\overline{B}+A\,\overline{B}+A\,\overline{C}+C=\overline{B}+A+C.$

   2 marks for answer in simple form. 2 marks for method.

   b) See Fig 1.1 for K-map. This gives $\overline{Y}=\overline{X_1}+X_2\,\overline{X_0}$. So (by DeMorgan's) $Y=X_1(\overline{X_2}+X_0)$.

   Must be in POS form for any marks. 2 marks for functional equivalence to minimal answer. 2 marks for minimal answer.

   c) $\overline{A+\overline{B}(\overline{C}+\overline{D})}=\overline{A}\,B(\overline{C}+\overline{D})=\overline{A}\,B\,\overline{C}+\overline{A}\,B\,\overline{D}$

   Must be in SOP form for any marks. 2 marks for answer. 2 marks for working.

   d) (i) 01001111 unsigned is 79, 11001111 unsigned is 207 (ii) 01001111 2c is 79, 11001111 2c is -49

   1 mark for each answer.

   e) $X^+=A+B$

   The simplified circuit could be anything computing this expression with fewer than three DFFs. One example is an OR gate feeding into the D input of a single DFF. Another is to keep the MUX in the question but move the three DFFs to the MUX output, replacing them with a single DFF as a result.

   2 marks for Boolean expression. 2 marks for simplified circuit.

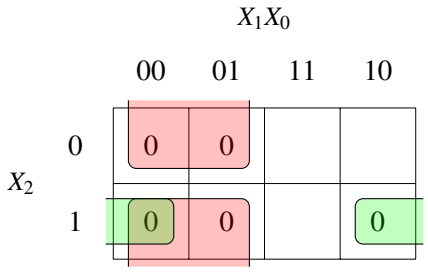


Figure 1.1: K-map

f) See Figure 1.2. 3 marks for each row: 1 mark for AD*, Y and CIN.

Operation	AD1(2:0)	AD2(2:0)	AD3(2:0)	Y(7:0)	Cin
R3 := R4 + R5	3	4 (or 5)	5 (or 4)	0	0
R2 := R0 − R1	2	0	1	255	1
R6 := 2 * R6	6	6	6	0	0

Figure 1.2: Q1f answers

g) See Figure 1.3. One mark for every value and address.

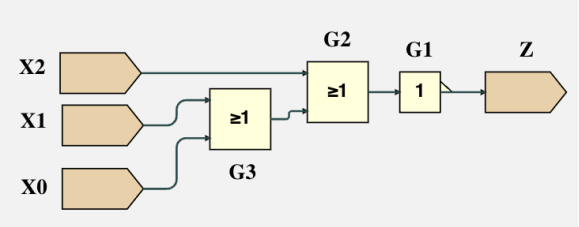
```
R0 = 0xFFFA
R1 =0xFF01
R2 = 0xFFE2
R3 = 0xFFFE
R4 = 0xC000
R5 = 0x8007
R7 = 0x0011
mem[6] = 4
mem[0xFFFF] = 8
```

Figure 1.3: Register and memory values

2. a) $-2^{n-1}+1$ up to $2^{n-1}-1$

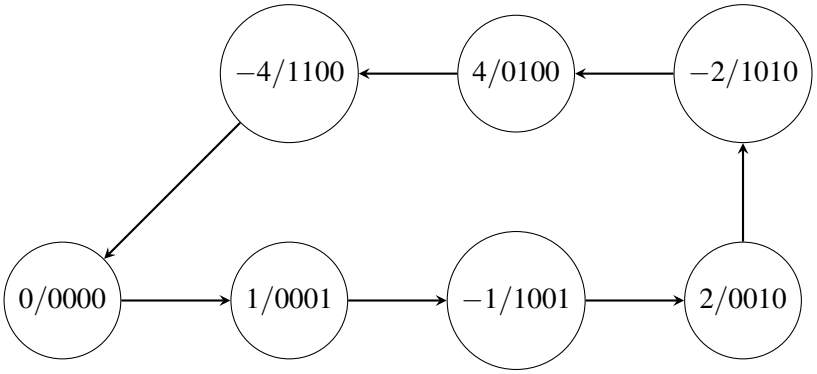
1 mark for lowest and 1 mark for highest representable value.

b) Figure below.



2 marks for any functional circuit using only AND, OR, NOT.

c) Moore diagram below.



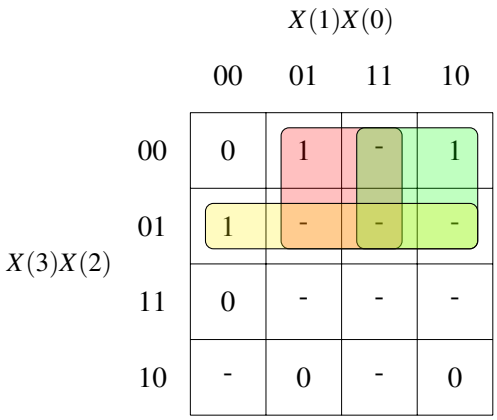
1 mark per correct labelling and outedge destination of each state.

d) K-maps are shown in Fig. 2.1.

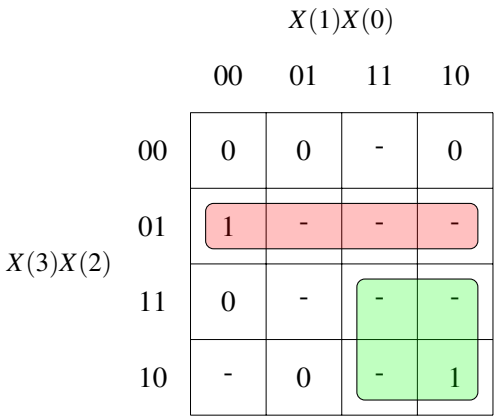
2 marks for each K-map, of which 1 is for functional correctness and 1 is for minimality in grouping.

e) K-maps are shown in Fig. 2.2.

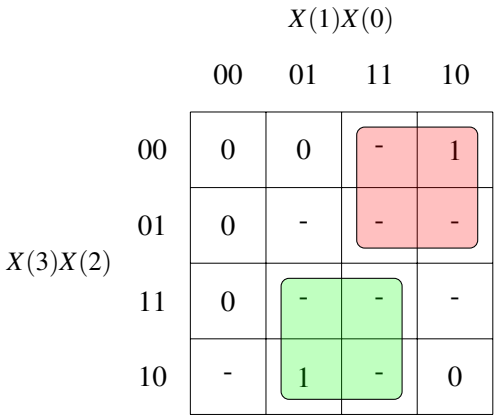
1 mark for each correct state-to-state K-map. 2 marks for each correct state-to-output K-map. For the latter, 1 mark for functionality and 1 mark for minimality.



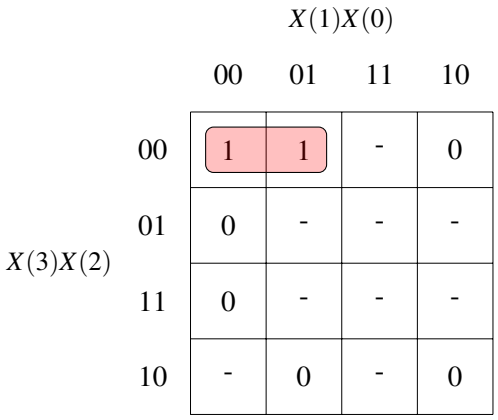
(a) $X^+(3)$



(b) $X^+(2)$

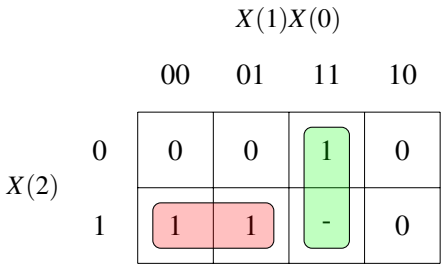


(c) $X^+(1)$

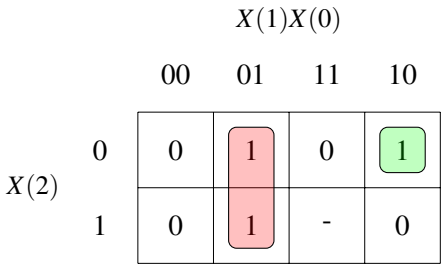


(d) $X^+(0)$

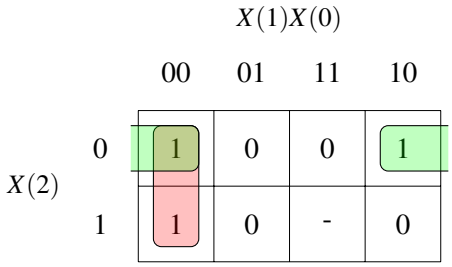
Figure 2.1: K-maps for Q2 Part (d)



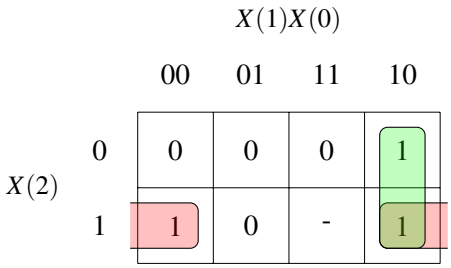
(a) $X^+(2)$



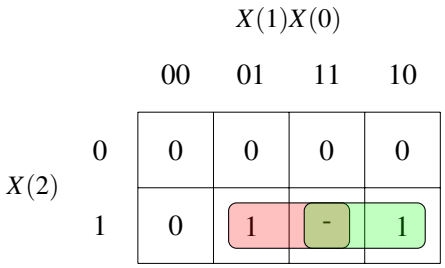
(b) $X^+(1)$



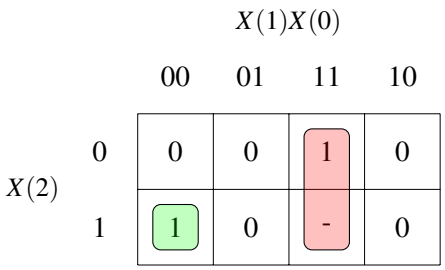
(c) $X^+(0)$



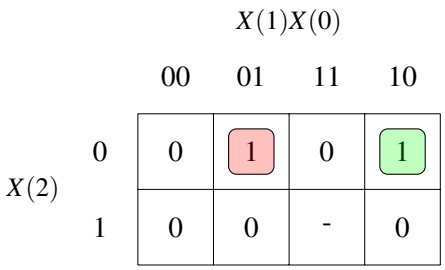
(d) $Y(3)$



(e) $Y(2)$



(f) $Y(1)$



(g) $Y(0)$

Figure 2.2: K-maps for Q2 Part (e)

3. a)

```
unsigned short r0,r1,r2,r3,r4;
r2 = 0;
r3 = r0;
r4 = r1;
while (r3) {
    if (r3 & 1) {
        r2 = r2 + r4;
    }
    r3 = r3 / 2;
    r4 = r4 * 2;
};
```

(1 mark each for *, / implementation of shifts. 1 mark for all correct, exact syntax does not matter).

[4]

b) The condition for no execution of the loop body, and `r2` on output zero, is `r3 = 0`. In that case in the assembly program the bit shifted out from R3 will be zero and therefore the jump to SKIP will happen, hence R2 will also remain 0.

[2]

c)

$$u(X) = 2^{15}X_S + X'$$

$$z(X) = -2^{15}X_S + X'$$

(1 mark)

$$u(R0) * u(R1) - z(R0)z(R1) = 2.2^{15}(R0_S R1' + R1_S R0') = 2^{16}(R0_S R1' + R1_S R0')$$

The difference is a multiple of 2^{16} therefore bits (15:0) of the signed and unsigned products must be the same. (2 marks).

R3 is right shifted in order to extract its bits in order shifting in a zero so that when all non-zero bits have been used the loop will end. Treating it as a signed number would mean using `ASR` and possibly shifting in 1s, in which case the loop would never terminate. (1 mark).

[4]

d) In MULTIPLY persistent registers R3,R4 are written and must be stored on the stack. There is no call to another subroutine so R7 need not be stored on the stack. The stack is downwards-growing empty. See Figure 3.1 for the code. (2 marks each for the entry and exit segments. 4 marks for SUMOFSQUARES).

[8]

e) See Figure 3.2. 1 mark for SP, 1 mark for each register number. 1 mark for correct order on stack.

[4]

f) After n loop iterations, at the CMP instruction, the initial value of R3 (R0) is divided by 2^n. The largest power of 2 in R0 is 2^m. Therefore R3 will first be zero after the $m+1$ iteration.

Inside the loop, the JCC instruction is taken, resulting in a loop time of 5 cycles, if the LSB of R3 at the start of the loop is 0. Otherwise the JCC is not taken, and the loop time is 6 cycles. The k 1's in R0 (and R3) correspond to k JCC SKIP jumps not taken, and $m+1-k$ JCC jumps taken.

EEP1 architecture. The program time from START is 3 cycles more than the loop time time which $3+6k+5(m+1-k) = k+5m+8$. (2 marks). Pipelined architecture. The number of stalling jumps is $m+1-1 = m$ (for the number of JNE jumps taken) plus $m+1-k$ (for the number of JCC jumps taken). The number of stall cycles is double this. Total throughput in Mhz is therefore:

$$\frac{100(k+5m+8)}{(k+5m+8)+(4m+2-2k)} = \frac{100(k+5m+8)}{9m-k+10}$$

(4 marks for EEP1 time. 2 marks for throughput).

[6]

```
MULTIPLY        STR R4, [R6]
                STR R3, [R6,#-1]
                SUB R6, #2
START           MOV R2, #0
                MOV R3, R0
                MOV R4, R1
LOOP            LSR R3, R3, #1
                JCC SKIP
                ADD R2, R2, R4
SKIP            LSL R4, R4, #1
                CMP R3, #0
                JNE LOOP
                ADD R6, #2
                LDR R4, [R6]
                LDR R3, [R6,#-1]
                RET

SUMOFPRODUCTS   MOV R3, R2
                JSR MULTIPLY
                MOV R0, R3
                MOV R1, R3
                MOV R3, R2
                JSR MULTIPLY
                ADD R0, R2, R3
```

Figure 3.1: Q3d answer

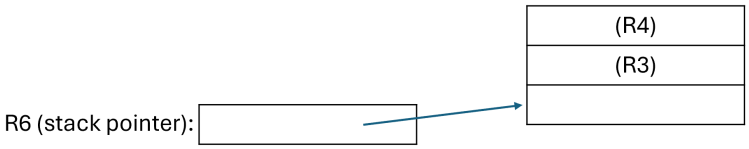


Figure 3.2: Q3e Pointer diagram of stack contents